

高级语言程序设计

实验报告

南开大学 计算机伯苓班

姓名 林永盛

学号 2511107

2026 年 5 月 9 日

目录

高级语言程序设计大作业实验报告	3
一. 作业题目	3
二. 开发软件	3
三. 课题要求	3
四. 游戏玩法与亮点介绍	1
1. 游戏玩法	1
(1) 核心机制	1
(2) 操作与模式	2
(3) 肉鸽成长系统	2
2. 核心亮点	2
(1) 差异化的角色与流派设计	2
(2) 动态难度与阶段递进	2
(3) 操作手感	2
(4) 视听动效反馈	3
五. 主要流程	3
1. 整体流程与系统架构	3
2. 核心算法与面向对象设计	3
(1) 多态与虚析构机制的广泛应用	4
(2) 策略模式构建武器系统	4
(3) AI 寻路与包围盒 (AABB) 碰撞检测	4
六. AI 工具使用披露	5

1. 使用的 AI 工具.....	5
(1) Gemini 3.1 Pro	5
(2) 豆包	5
2. 核心应用场景与具体用途.....	5
(1) 底层架构与设计阶段	5
(2) 算法攻坚与性能优化阶段	5
(3) Bug 追踪与健壮性提升阶段	6
七. 总结与收获.....	6
1. 内存管理的重要性.....	6

高级语言程序设计大作业实验报告

一、作业题目

熊出没类幸存者游戏：基于 C++ 与 EasyX 框架开发的面向对象双人同屏或单人类幸存者肉鸽生存游戏。

二、开发软件

1.IDE: Visual Studio 2022

2.图形库: EasyX Graphics Library

三、课题要求

1. 使用 C++ 语言完成一个图形化的小程序，图形化平台不限（本项目采用 EasyX）。
2. 程序内容主题不限，看重创新、创意。
3. 运用面向对象（OOP）编程思想。
4. 代码通过 Gitee/GitHub 托管，体现版本迭代。
5. 录制 B 站项目讲解与演示视频。

四、游戏玩法与亮点介绍

1. 游戏玩法

(1) 核心机制：本游戏是一款基于 C++ 与 EasyX 开发的类幸存者肉鸽

生存游戏。玩家需要操控《熊出没》IP 中的经典角色，在源源不断袭来的敌人包围中极限生存。角色会自动使用武器进行攻击，玩家的核心任务是抓准时机释放技能和利用走位躲避敌群。

(2) 操作与模式：游戏原生支持单人游戏与本地双人同屏合作游戏。双人模式下，两名玩家分别通过键盘左侧的 WASD 键与右侧的方向键进行完全独立的控制，相互配合抵御怪海，极大增加了游戏的社交属性与趣味性。

(3) 肉鸽成长系统：击倒敌人后会在场景中掉落经验球，玩家靠近时会触发磁吸插值算法自动拾取。经验槽积满后角色将升级，并弹出经典的“随机三选一”强化面板，供玩家自由搭配流派，以应对随时间推移而不断变强的敌群。

2. 核心亮点

(1) 差异化的角色与流派设计：游戏摒弃了同质化的换皮，精心设计了三名机制截然不同的经典角色。熊大定位为重装战士，血量高、拥有极坐标运算生成的两颗岩石进行圆周环绕护体；熊二定位为敏捷刺客，移速高、拥有基于正弦波动轨迹的心跳式蜂蜜脉冲武器；光头强定位为高爆发射手，伤害高、血量低、装备基于对象池技术管理的高频弹幕猎枪。

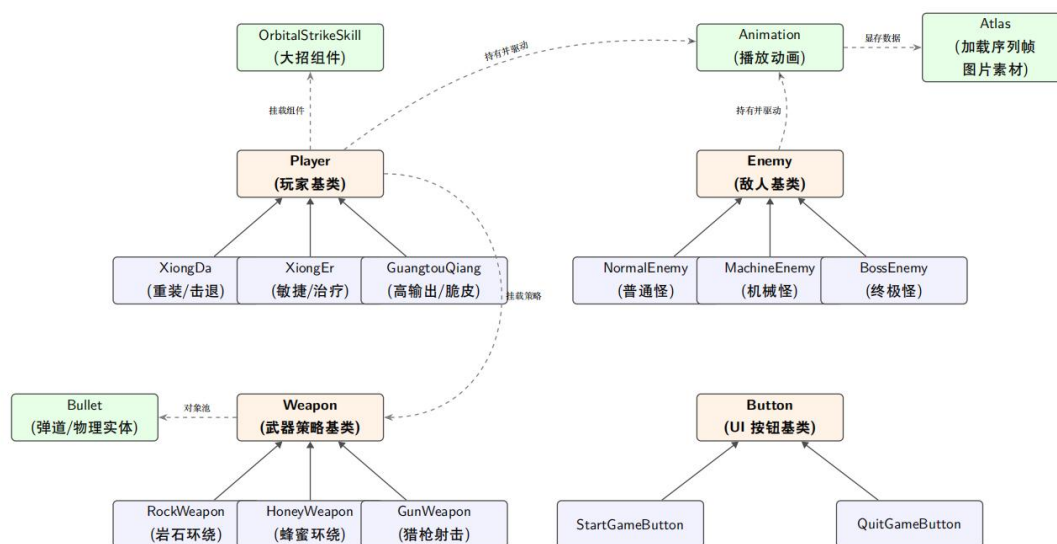
(2) 动态难度与阶段递进：游戏拥有完善的阶段调度器。随着玩家存活时间的增加，敌人会从基础的普通怪，升级为拥有更高血量和移速的“机械怪”。在最终阶段登场的 Boss 更是配备了定时召唤小怪增援的专属机制，持续给玩家带来压迫感。

(3) 操作手感：底层碰撞检测摒弃了粗糙的点阵判断，采用了 AABB 矩形碰撞盒算法。为了优化割草游戏的走位体验，特意将实体判定区向内收缩

了 15 像素，赋予了玩家在密集怪海中极限微操的空间。

(4) 视听动效反馈：借助双缓冲渲染技术解决了画面闪烁问题。同时加入了海量视觉反馈：怪物受击时的短暂闪烁与物理击退、动态飘出的实时伤害数字、自适应的同屏双人独立血槽，以及角色阵亡时 3.5 秒的临终特效，提供了极其解压爽快的“割草”视听体验。

五、主要流程



1. 整体流程与系统架构

本项目采用经典的游戏主循环架构，通过 `BeginBatchDraw()` 与 `FlushBatchDraw()` 实现双缓冲渲染，彻底消除画面闪烁。

游戏分为两个主要状态：**主菜单状态**（UI 交互、选人、模式切换）与 **战斗状态**（实体生成、AI 追踪、物理碰撞、经验结算、动画渲染）。支持单人游玩与本地双人同屏合作（键盘左侧 WASD 与右侧方向键）。

2. 核心算法与面向对象设计

本项目实践了 C++ 的面向对象特性，主要类关系与架构如下：

(1) 多态与虚析构机制的广泛应用

- **UI 系统**：定义了纯虚基类 `Button`，包含虚函数 `ProcessEvent()`、`Draw()` 和 `virtual void OnClick() = 0;`。派生出 `StartGameButton` 和 `QuitGameButton`。在主循环中，利用 `vector<Button*> menu_buttons` 统一管理，通过基类指针实现多态调用，消除了硬编码。
- **敌方系统**：定义了 `Enemy` 基类，派生出 `NormalEnemy`（普通怪）、`MachineEnemy`（高频高伤怪）和 `BossEnemy`（最终 Boss）。其中 `Boss` 实体通过 `override` 重写了 `Move()` 函数，在继承基础追踪算法的同时，引入了定时召唤小怪的专属机制。

(2) 策略模式构建武器系统

为了解决不同角色攻击方式差异巨大的问题，设计了 `Weapon` 抽象基类。

- **基类**：`Weapon` 统一定义了 `Update()` 和 `Draw()` 接口，并管理 `vector<Bullet> bullets` 弹药池。
- **派生类**：
 - `RockWeapon`（熊大专属）：极坐标运算实现两颗岩石的圆周环绕。
 - `HoneyWeapon`（熊二专属）：引入正弦波动实现类似心脏跳动的椭圆脉冲收缩轨道。
 - `GunWeapon`（光头强专属）：基于对象池模式，管理 20 发子弹的线性射击与出屏自动内存回收。

玩家基类 `Player` 仅持有 `Weapon* current_weapon` 指针，实现了角色本体与攻击逻辑的解耦。

(3) AI 寻路与包围盒（AABB）碰撞检测

- **AI 索敌**：遍历场上存活的 Player，利用勾股定理计算最短欧几里得距离，锁定仇恨目标。利用向量归一化获取方向系数并乘以移速，实现匀速迫近。
- **碰撞检测**：摒弃了简单的点碰撞，采用了基于实体宽高的 AABB (Axis-Aligned Bounding Box) 矩形碰撞盒算法，并向内收缩 15 像素作为硬核判定区，提升了玩家操作手感。

六、AI 工具使用披露

按照课程大作业要求，在此对本项目开发过程中的 AI 工具使用情况进行披露。本项目秉持“独立思考、工具辅助”的原则，主要借助 AI 进行架构优化与问题排查。

1. 使用的 AI 工具

- (1) Gemini 3.1 Pro (架构设计与算法优化)
- (2) 豆包 (素材生成)

2. 使用位置与具体用途

(1) **底层架构与设计阶段**：项目伊始，我就确立了“分文件编写”与“面向对象 (OOP)”的基础架构。在此框架下，为了进一步提升武器与角色模块的拓展性，我利用 AI 探讨了高级设计模式，成功将“策略模式” (Weapon 接口及其派生类) 与“工厂模式” (CreatePlayer 逻辑) 融入到我的 OOP 体系中。

(2) 算法攻坚与性能优化阶段：

- **对象池机制优化**：在光头强角色的子弹管理中，借助 AI 优化了无序数组的 Swap-Pop 删除策略，将内存回收的时间复杂度降为 $O(1)$ 。
- **缓动特效实现**：利用 AI 辅助推导经验球的磁吸缓动插值算法，以及利用三角函数 ($\sin/\cos$) 实现武器运转与特效呼吸灯的极坐标轨迹计算。

(3) **Bug 追踪与健壮性提升阶段**：协助排查并修复了多处涉及 C++ 底层机制的隐蔽 Bug，例如：修复了双人模式下玩家阵亡后武器特效因未清空而残留的“幽灵特效” Bug；定位了基类缺乏虚析构函数导致的内存泄漏隐患；修复了受击飘字机制中伤害溢出血量上限的视觉表现 Bug。

3. 其他辅助用途

资源生成：利用 AI 生成了部分素材图、背景原画。

七、总结与收获

1. 内存管理的重要性

本项目大量使用了 new 分配堆内存。为杜绝内存泄漏，我在 main 循环结束的收尾阶段，统一遍历并 delete 了所有产生的对象指针，随后彻底 clear 容器，实现了资源的安全回收，加深了对 C++ 内存管理的实战理解。